CLIQ Data Acquisition Library Overview

CLIQ is a Rust-based data acquisition program designed to work with CAEN VX2740 digitizers (used in the COH-Ar-750 experiment)[1]. It wraps the CAEN FELib C API with Rust code and provides a text-based user interface (TUI) for monitoring data-taking. The software continuously reads out waveforms from the digitizers and writes them to HDF5 files, handling multi-threaded data collection, online processing (like zero-suppression), and file management. This overview explains the design and operation of CLIQ's core components – focusing on the main program (`main.rs`), the TUI controller (`tui.rs`), and the HDF5 writer (`writer.rs`) – so that a developer familiar with C++ (but new to Rust) can understand and contribute to the project. Comparisons to C++ constructs are provided to help bridge the concepts.

Main Program Setup (`main.rs`)

The entry point of the application is the `main` function in **`src/main.rs`**, which is responsible for initial setup and then handing off control to the TUI-driven runtime[2]. The main tasks performed here are analogous to what a C++ program's `main` would do before entering an event loop or main processing thread:

- **Command-line Parsing:** CLIQ uses the Rust `clap` library to parse command-line arguments. A custom `Args` struct (using `#[derive(Parser)]`) defines options like the path to the configuration file (`--config`) and an optional number of runs to execute[3]. This is similar to using a library like Boost.Program_options or `getopt` in C++ to handle command-line arguments.

- **Configuration Loading:** After parsing args, `main` loads the run configuration from a TOML file. It calls `Conf::from_file(&args.config)` to parse the TOML config into a `Conf` struct[4]. This utilizes the third-party **Confique** library for configuration management[5]. In C++ terms, this is akin to reading a configuration file (perhaps via a library like Boost.PropertyTree or JSON/YAML parsers) into an in-memory config object.

- **Device Initialization:** Using the config, `main` prepares the list of digitizer boards to use. It iterates through each board address (e.g. a VX2740 connection URI) provided in the config and opens the board via `felib_open`, a wrapper around the CAEN FE (Front-End) library's open function[6]. In Rust, external C functions are wrapped as `felib_open` and friends (likely defined in `felib.rs`), and error handling is done with Rust's `Result`. This would be analogous to calling the CAEN API directly in C/C++ to obtain device handles. The opened device handles (simple numeric identifiers from the C API) are stored along with board

IDs in a `boards` vector[7]. Each entry is a tuple `(board_index, device_handle)`.

- **Logging Setup:** The program initializes a log file (`daq.log`) and sets up logging with timestamp formatting using the **simplelog** crate[8]. This corresponds to configuring a logger in C++ (for example, using `spdlog` or `log4cxx`) to write messages with timestamps to a file. The log level is set to `Debug`, meaning the program will log detailed runtime information.

- **Launching the TUI:** With configuration and devices ready, `main` sets up the terminal UI environment and enters the main loop managed by the TUI. This is done by calling `ratatui::init()` to initialize the terminal (preparing it for drawing text-based UI) and then constructing a `Tui` object with all the necessary state[9]. The `Tui::new` function takes the configuration, the list of board handles, the optional max run count, and the config file path, and returns a `Tui` instance. Immediately after, `main` calls `Tui.run(&mut terminal)` to start the TUI-driven workflow[10]. From this point on, the TUI module takes over program control; as noted in the documentation, "once the TUI begins to run that's all there is to `main.rs`"[11]. In C++ terms, this is similar to instantiating a controller class and calling its run method (for example, entering a GUI/TUI event loop or main processing loop), after which the `main` function mostly waits for that to finish.

- **Program Shutdown:** When the TUI loop exits, `main` will restore the terminal state (via `ratatui::restore()`) and print a friendly goodbye message (`"TTFN!"`) before returning[12]. This cleanup ensures that the terminal settings (like raw mode) are reset back to normal, analogous to making sure any UI library cleans up console settings in C++. The use of Rust's `Drop` traits and safe abstractions means resources like open file handles or device connections are closed appropriately when their objects go out of scope, but as an extra measure, the code explicitly closes device handles in the TUI module when ending (discussed below).

In summary, `main.rs` handles configuration, logging, and device setup, then delegates to the TUI. These steps should look familiar from a C++ perspective: parse config, initialize hardware, then enter the main loop.

TUI Controller and Program Flow (`tui.rs`)

The **TUI (Text-based User Interface)** module, in **`src/tui.rs`**, contains the core runtime logic and state machine of CLIQ. It is analogous to the primary control loop in a C++ program (or the main window/controller in a GUI app) that manages the overall flow, except here the interface is a text UI drawn in the terminal. This module is where

most of the program's complexity resides[13], as it must coordinate multiple threads for data acquisition and writing, handle user input, and update the on-screen display.

TUI Structure and State: The `Tui` struct holds the state needed throughout the data acquisition runs[14]. Key fields include the current run number, the campaign number (a higher-level grouping of runs), the parsed configuration (`Conf`), the list of opened board handles, and runtime status counters (for events, buffer usage, errors, etc.). It also tracks whether the user requested to exit (`exit` flag) and supports an optional limit on number of runs (`max_runs`) from the command-line. Think of `Tui` as a C++ class that encapsulates global state instead of using a bunch of globals – it carries configuration and status, and methods on `Tui` manage the run loop.

Run Loop: The main entry point here is `Tui::run(&mut terminal)`, which implements the continuous run loop. When `run()` is called, the program will repeatedly start data acquisition runs and either loop indefinitely or for a fixed number of runs, until a quit condition is met[15]. Each iteration of this loop corresponds to one "run" of data taking (with a duration specified in the config). The high-level steps in each run cycle are:

1. **Hardware Reset and Configuration:** At the beginning of a run, the TUI resets each digitizer board and applies the configuration settings to them[16][17]. This is done by calling functions (from `felib.rs` and `utils.rs`) like `felib_sendcommand(..., "/cmd/reset")` and `configure_board` for each board, and `configure_sync` to set up multi-board synchronization (clock and trigger sync)[16]. In C++ one would similarly call the vendor's API to reset and configure devices; here those calls are wrapped in safe Rust functions. Logging is generated to note that the boards have been reset and configured (`info!("Reset and configured digitizer(s)")`).

2. **Spawning Data Threads:** The TUI then prepares to start data acquisition by spawning a set of background threads. **One thread per board** is spawned to handle data readout from that digitizer, and **one thread for event processing** is spawned to aggregate and write out events[18]. Rust's thread spawning (`std::thread::spawn`) is analogous to creating `std::thread` in C++. Each board's thread runs the `data_taking_thread` function, while the event processing thread runs the `event_processing` function. The `Tui.begin_run()` method sets up these threads and returns handles to manage them[18]. Notably, the threads communicate via **channels** for thread-safe message passing: CLIQ uses a multi-producer, single-consumer channel from the Crossbeam library (`crossbeam_channel::unbounded`) to send events from each board thread to the single event-processing thread[19]. This pattern will be familiar to C++

developers as well – it's like having producer threads push data into a concurrent queue protected by a mutex/condvar, and a consumer thread pop from it. Crossbeam's channel abstracts those details (it's conceptually similar to a thread-safe queue or a pipe).

3. **Thread Coordination (Start/Stop):** Before data starts flowing, the threads use synchronization primitives to coordinate start times and handle shutdown signals. In Rust, an `Arc<Mutex<...>>` with a `Condvar` is used to synchronize all board threads: each board thread waits on a condition variable until all boards are ready, then the main thread signals them to begin acquisition simultaneously[20] [21]. This is analogous to using `std::mutex` and `std::condition_variable` in C++ for thread sync. Specifically, each board's thread initializes the digitizer's "endpoint" for data readout (setting up the digitizer's internal data format and arming it) and then increments a shared counter. The main thread waits until all boards have finished this setup (a condition variable `endpoint_configured` is notified by each board thread)[22][23]. Once ready, the main thread calls a start command (CAEN's software start acquisition) on one board and signals an `acq_start` condition variable to release all the board threads at once[24]. This mechanism ensures all boards begin taking data in sync. The use of an atomic boolean flag (`shutdown: Arc<AtomicBool>`) is another concurrency element: it provides a shared signal that can be set to tell threads to stop. This is similar to using an `std::atomic<bool>` or a global flag in C++ to request thread shutdown, but with the safety of `Arc` (Rust's thread-safe reference-counted pointer) to allow sharing the flag across threads.

4. **Data Taking Threads:** Once released, each board thread enters a tight loop reading events from its digitizer. Internally, `data_taking_thread()` calls the CAEN FELib function to read data (wrapped as `felib_readdata`) in a loop[25]. If a read is successful (`Success`), it obtains an `EventWrapper` object containing the waveform and metadata, packages it with the board ID into a `BoardEvent`, and sends it through the channel to the event processing thread[26]. To avoid constantly allocating new event buffers, it uses a technique analogous to reusing buffers in C++: it calls `std::mem::replace` to swap out the used `EventWrapper` with a fresh one, reusing the allocated memory for subsequent reads[27]. If the read returns a timeout (no data yet), the thread simply continues (like `continue` in C++ loop), and if a stop or error is indicated, it breaks out. Each board thread runs independently, streaming events as fast as the hardware provides, until a shutdown is signaled or the device stops. These threads are "pretty simple" as the README notes[19] – essentially looping on data acquisition and forwarding events. In C++, you would implement something similar with a thread that calls a device API in a loop and enqueues results.

5. **Event Processing Thread:** The single event-processing thread is the consumer that receives all `BoardEvent`s from the channel. Its job is to **buffer and align events**, then write them to disk via the HDF5 writer[28]. Each `BoardEvent` includes a board ID and one event's data (waveforms and associated info). The thread maintains a queue (buffer) for each board, storing incoming events. It ensures that for a given trigger number, it has collected the corresponding event from every board before writing. This alignment step is important in multi-board setups: it matches events by their trigger IDs across boards. If one board is lagging or an event is missing, the code can detect misalignment. Specifically, when all queues have at least one event, the code compares the trigger IDs; if they don't match, it uses a function `align_queues` to drop or realign events so that the fronts align[29][30]. It also counts any misaligned or dropped events for reporting. Once events are aligned, the thread removes the front event from each board's queue (forming a set of concurrent events) and writes them out together. It sends a small `RunInfo` summary back to the main thread via a separate channel (tx_stats) containing stats like the total size of events just processed, current channel queue length, and counts of misalignments or drops[31][32]. The TUI uses this to update on-screen counters (e.g., number of events, buffer backlog, etc.).

6. **Zero Suppression (online):** Before writing, the event-processing thread applies zero suppression to waveform data if enabled. In CLIQ, zero suppression is configurable and works by zeroing out waveform samples that don't contain a significant pulse, to reduce data size. The code uses the `zero_suppress()` function to process each event's waveforms in place[33][34]. It calculates the baseline and standard deviation for the first N samples of each channel, then checks for pulses exceeding a threshold (with either positive or negative edge as specified) within a sliding window[35][36]. If no pulse is found or the noise is too high (stdev above a limit), it sets all samples to zero for that channel[37][38]. This operation is performed using Rust's **ndarray** library with parallel iterators (`into_par_iter()`), meaning channels are processed in parallel using multiple CPU cores[39]. For a C++ developer, this is analogous to using multi-threading (or vectorized operations) to iterate over array data; one might use OpenMP or C++17's `<execution>` parallel algorithms to achieve a similar parallel loop. CLIQ's zero suppression is done *in real-time as data is streaming*, which in C++ would typically be handled by a similar loop over channels with optional parallelization.

Importantly, CLIQ implements a **prescale for unsuppressed events**: the config can specify a fraction of events that should be left unsuppressed for validation. In the code, `zs_prescale` (e.g., if set to 10) means every 10th event is *saved twice*: once normally

suppressed and once with no suppression[40][41]. The event-processing thread achieves this by cloning the event when the count hits the prescale interval and marking one copy as unsuppressed[41][42]. This way, most events have zeroed-out baseline data, but periodically a full waveform is kept for comparison – a technique a C++ program might also use (e.g., keep 10% raw events for monitoring). The code uses a boolean flag `zero_suppressed` in the `BoardEvent` to note whether a given event was zero-suppressed or not, and writes that to the file as a dataset.

1. **Writing to File:** Once a batch of aligned events is prepared and possibly zero-suppressed, the event-processing thread appends them to the HDF5 file via the `HDF5Writer` (details in the next section). It calls `writer.append_event(...)` for each event[43][44]. The writer buffers a group of events (across boards) in memory and flushes to disk when the buffer is full or when ending the run. The processing thread continues this loop of receive -> align -> suppress -> write until a shutdown signal is detected or the run ends. When the run duration expires (timed by `run_duration` from config), the TUI will set the `exit` status to `Timeout`, which causes the main loop to break out and begin the run termination procedure[45]. The shutdown flag is then set so that the board threads and event thread break their loops as well[46][47].

2. **User Input Handling:** Meanwhile, the main TUI thread is not idle. It continuously updates the terminal UI and polls for user key presses. Every iteration of the run loop, the TUI draws the interface (showing updated stats) and then checks for input events using `crossterm` (the backend for ratatui)[48][49]. If the user presses **`q`**, the TUI sets its exit status to `Quit`[50]. This will break out of the run loop at the next opportunity, initiating shutdown (closing threads and devices). This cooperative multitasking – where the main thread handles UI and control logic, while background threads handle I/O – is analogous to having a main GUI thread and worker threads in a C++ program, or an event loop with worker threads feeding it data. The UI is redrawn at a regular interval (the code uses a 1-second tick via Crossbeam's `tick` channel to trigger periodic refreshes) as well as on certain events, so the display (counters, statuses) remains current.

3. **Terminal UI Layout:** The user interface is built with the **ratatui** library (a Rust TUI framework similar to C++ NCurses or ImGui for console). The screen is split into regions: a top section for global run status and instructions, and a bottom section showing per-board status[51][52]. For each board, metrics like trigger rates and dead time are fetched via calls to `felib_getvalue` on various parameters and displayed in a text widget[53][54]. This way, the operator can see in real time the digitizer's performance counters (similar to how a C++ program might use NCurses to print such info in a loop). The top status bar shows the

campaign and run number, elapsed time, total events collected, data rate (MB/s), current event buffer backlog, and counts of misaligned or dropped events[55][56]. All text is styled (e.g. numbers in yellow) for readability. Pressing `q` is indicated on the UI ("Press <Q> to quit") as a reminder. If an error occurs (like misaligned events beyond tolerance), the TUI can overlay a **popup** message on the screen (`show_popup`)[57]. For example, if a data thread reports a `MisalignedEvents` error, the main thread sets `self.show_popup` with a message and draws it, prompting the user to acknowledge by pressing `q` to quit[58][59]. This is akin to showing a modal error dialog in a GUI – here done with text on a bordered block in the terminal. The program will then shut down after the user presses `q` since `exit` is set.

4. **Run Cycle Completion:** When a run completes normally (timeout reached), the TUI performs an orderly shutdown of that cycle. It signals the board threads to stop (sets the shutdown atomic flag), sends a disarm command to each board (`/cmd/disarmacquisition`) to stop acquisition in hardware, then joins all the threads (waits for them to finish)[60][61]. The threads return a `Result<(), DaqError>`; if any thread reported a `DaqError` (like data transit error), it handles it as described (popup message). After the event-processing thread is joined, CLIQ increments the run number and, if the program is set to do multiple runs, it will automatically start the next run without restarting the program[62][63]. The configuration in memory is reused for the next run (the same settings), but the program does create a fresh output file for the new run (with an incremented run number in the filename). This looping mechanism means the program can run continuously, splitting data into sequential runs. If the user had specified a maximum number of runs (`--runs`), the TUI checks after each run and will exit once that many runs have been completed[63]. It closes all board connections via `felib_close` when finishing for good[64][65].

From a C++ perspective, the TUI module's structure resembles an event-driven controller that spawns threads and uses synchronization to coordinate them. Concepts like threads, condition variables, atomic flags, and producer-consumer queues work the same way in Rust, though Rust's type system enforces safety (e.g., ensuring no data races on shared state by requiring `Send + Sync` traits for shared data). A developer used to C++11 threads will find the logic familiar: spawn threads, use mutex/condvar or atomics for sync, communicate via queue (channel), join threads at the end. Rust's ownership model means that the `Tui` struct cleanly owns the resources (board handles, config) and passes copies or handles to threads in a controlled way (Arc clones for shared flags, cloning the `Conf` for read-only use in threads, etc.), which prevents common C++ pitfalls like dangling pointers. Error handling is done with `Result` rather than exceptions – errors are bubbled up as needed (you see many functions return

`Result<(), DaqError>` and uses of the `?` operator to propagate errors). This is somewhat like using error codes or `std::optional`/`Expected` in C++ to handle errors without exceptions.

HDF5 Data Writer (`writer.rs`)

The **`HDF5Writer`** struct (defined in **`src/writer.rs`**) encapsulates all logic for creating and writing to the HDF5 output files. Its design is to efficiently handle high-volume waveform data by buffering writes and splitting output into manageable file sizes. In C++ terms, this is like a file-writing module/class that uses the HDF5 C/C++ library to create files, datasets, and write data, with internal buffers to batch writes for performance.

Key aspects of the HDF5Writer design:

- **File and Dataset Setup:** When a new run starts, the TUI's event-processing thread creates a new HDF5 file by calling `HDF5Writer::new(...)`. The `new` method opens/creates the file and sets up the structure: it creates a group for each board (named `"board0"`, `"board1"`, etc.) and within each group it creates datasets for **timestamps**, **waveforms**, **trigger IDs**, **flags**, **board fail status**, and **zero_suppressed mask**[66][67]. Each dataset is created with a maximum size corresponding to the maximum number of events per board (a configurable value, e.g. 7500 events)[68]. They use chunked storage and apply **Blosc compression** (Zstd) at a specified compression level for each dataset[69][70]. For example, the waveform dataset might be of shape `(max_events, 64, record_len)` and chunked in units of 50 events (the buffer size) for efficient writes[71][72]. In a C++ HDF5 implementation, one would similarly define dataspace dimensions, chunking, and compression filters when creating datasets. The `HDF5Writer` also initializes an HDF5 file attribute `"saved_events"` to track how many events have been written[73][74] (this attribute gets updated when the file is closed or rolled over).

- **In-Memory Buffers:** The writer allocates in-memory **buffers** for a certain number of events (by default, 50 events per board)[75]. The `buffer_capacity` (50) is chosen as a trade-off: instead of writing each event individually, which would be slow, events are accumulated and written in bulk. Internally, the `BoardData` struct holds arrays (using the ndarray data structure) to buffer timestamps, waveforms, etc., for up to 50 events before a flush[76][77]. Each board's data is buffered separately. This is conceptually the same as using intermediate arrays or `std::vector` in C++ to collect data and then writing a block to file once the buffer is full.

- **Appending Events:** The event-processing thread calls `HDF5Writer.append_event(board_id, timestamp, waveform_data, trigger_id, flags, fail, zs_mask)` for each event[43][44]. This method places the event's data into the in-memory buffers of the appropriate board. The implementation checks that adding this event won't exceed the dataset's max capacity (`max_events_per_board`). If it would, that triggers a **file rollover** (described below)[78][79]. Otherwise, it copies the data into the buffer arrays and increments a buffer counter. When a board's buffer count reaches the capacity (50 events), the writer automatically **flushes** that buffer to disk[80]. Flushing means writing the buffered data slice into the HDF5 dataset at the correct index offset and resetting the buffer for reuse[81][82]. For example, if 50 new events are buffered, it will write a 50x64xN slice into the waveform dataset at the position corresponding to the current event count. The `current_event` index is then updated. This approach is similar to using `H5Dwrite` in C to write a hyperslab of an HDF5 dataset for a batch of events, and using an offset that tracks how many events have been written so far.

- **File Rollover:** Each output file is limited in how many events it contains (per board) to keep file sizes reasonable. The CLIQ README notes that if a file holds more than **~7500 events per board**, the writer will "roll over" to a new file[75]. The exact threshold is set by `max_events_per_board` in the config (7500 by default). When `append_event` detects that adding a new event would exceed this maximum, it returns a specific error which triggers the `HDF5Writer.rollover()` method[78]. Rollover closes the current file and opens a new file with an incremented sub-run index in the filename[83][84]. For example, if the base filename is `run000001_00.h5` (run 1, subrun 0), the next file becomes `run000001_01.h5`, then `_02.h5`, and so on[84][75]. The writer carries over any events that were in buffers at the time of rollover into the new file: it takes the partial buffers from the old file's `BoardData`, creates new datasets in the new file, and writes those buffered events into the new file so no data is lost[83][85]. This is done by an `append_buffer` helper that writes a given buffer en bloc to a fresh dataset[86][87]. The `subrun` counter in `HDF5Writer` tracks these sequential files within the same run. This mechanism is comparable to log file rotation in systems programming or splitting data files in C++ after a size limit: e.g., a C++ program might close the current output file and open a new one when a certain number of events have been written.

- **Compression and Threads:** The HDF5 writer leverages HDF5's support for compression. It uses the Blosc compression library (with Zstd codec) for all datasets, as seen by calls like `.blosc_zstd(compression_level, shuffle)` when creating datasets[69][88]. Additionally, it sets the number of threads for

Blosc (`blosc_set_nthreads`) based on a configuration parameter[89][90]. This means the compression of chunks can happen in parallel using multiple CPU cores (helpful for large waveforms). A C++ implementation using HDF5 with Blosc would similarly configure the Blosc filter and possibly allow multi-threaded compression. The `compression_level` (e.g., 5 or 9) can be tuned, and `n_threads` might be set to the number of CPU cores to speed up compression.

- **Flush and Close:** When a run ends or the program is shutting down, the remaining buffered events (if any) need to be flushed to disk. The event-processing thread ensures this by calling `writer.flush_all()` when exiting its loop or when the channel closes[91][92]. `flush_all` writes out any events still in the buffers of each board and updates the `"saved_events"` attribute in the file to the total number of events written[93][94]. Updating this attribute is akin to writing a summary variable to the file (e.g., in C++ one might call H5Awrite on an attribute). Finally, dropping or destroying the `HDF5Writer` will close the file (the HDF5 library takes care of releasing resources when the File object goes out of scope, similar to H5Fclose in C).

The writer's design is modular: it hides the low-level HDF5 operations behind a convenient API (`append_event` etc.), so the rest of the code doesn't need to deal with HDF5 details. It's conceptually like a C++ class that uses the HDF5 C API internally, exposing simple methods to add data. For a developer new to Rust, it's worth noting that the HDF5 crate's API uses Rust's ownership and error handling mechanisms, but beneath that, it's calling the HDF5 C library. The use of `anyhow::Result` in `writer.rs` means that many errors (file I/O, HDF5 errors) are propagated as `Result<..., Error>` instead of throwing exceptions, which is analogous to returning error codes in C++ or using try-catch if the HDF5 C++ API throws. The code explicitly checks for the "Maximum number of events reached" condition by examining the error message string[78] – a somewhat crude but effective way to detect the rollover condition. In C++ you might instead use a specific error code or a boolean return to trigger a new file.

Data Organization: The output HDF5 files have a specific layout (documented in the README) that developers should know when working on the code[95]. Each board's data lives in a separate group (`/board0`, `/board1`, etc.). Within each group, the datasets are:

- `timestamps` (1D array of 64-bit timestamps for each event),
- `waveforms` (3D array of shape num_events × 64 × record_len, storing all channels' waveforms),
- `triggerids` (trigger ID numbers for each event),
- `flags` (16-bit flags from the digitizer for each event),

- `boardfail` (boolean indicating if the board was in a failure state for that event),
- `zero_suppressed` (a mask or indicator if the event's waveform was zero-suppressed)[95].

This structure is created in `create_boards()` during file init, and the code writing data (in `BoardData.flush`) maps directly to these datasets. For instance, writing the waveform buffer uses `waveforms.write_slice(...)` with the appropriate index range[96][97] to place a batch of events into the dataset. If you've used HDF5 in C++, this corresponds to creating datasets with H5Dcreate and writing with H5Dwrite using hyperslab selections.

Overall, the `writer.rs` component ensures that high data rates can be handled – it buffers 50 events at a time, compresses data on the fly, and splits output into multiple files to avoid any single file becoming too unwieldy. This design is critical for performance, much like in C++ one would buffer I/O and possibly use multiple threads for compression or I/O.

Comparison to C++ Concepts

For a C++ developer reviewing CLIQ's Rust code, here are some analogous concepts and design patterns to help in understanding and contributing to the project:

- **Ownership and Resource Management:** Rust's strict ownership means that resources (like the board handles, which are numeric but represent external connections) are tied to variables and cannot be accidentally leaked or used after free. In C++, one must be careful to close file descriptors or free memory; in Rust, these are largely managed by RAII (the same principle as C++'s destructors, but enforced by the compiler). For example, the board handles are closed explicitly with `felib_close` when the TUI ends[64], similar to calling a close function in a C++ destructor or after use.

- **Error Handling:** Rust does not use exceptions. Functions that can fail return `Result` types, and the `?` operator is used to propagate errors. In the code, you'll see many functions returning `Result<(), DaqError>` or `anyhow::Result<T>`. This is analogous to methods returning error codes in C++ or using try-catch around library calls. When reading the code, mentally translate `?` as "if error, return early" – akin to checking a return code and returning from the function on failure.

- **Threads and Synchronization:** Rust's threading model is very similar to C++11's `std::thread`. The use of `Arc<Mutex<T>>` corresponds to `std::shared_ptr<std::mutex>` (with some differences, but conceptually both

allow shared ownership of a lock). `Arc<AtomicBool>` is like a shared atomic flag (since C++ atomics can be global or shared by pointer). Condition variables in Rust play the same role as `std::condition_variable` in C++. One difference is that Rust closures capture variables by value or reference when spawning threads; in CLIQ, they clone Arcs and the config for each thread so that each thread has its own copy or reference. A C++ implementation might capture pointers or references to shared data in a lambda passed to `std::thread`. The careful use of these primitives in CLIQ ensures no data races: for example, the counters updated by the event thread are sent via channel instead of being written to shared memory directly, avoiding locks on the fast path.

- **Channels vs Queues:** The Crossbeam channel used for events (and another for stats) is conceptually the same as using a concurrent queue protected by a mutex and condition variable. In modern C++, one might use a library like Intel TBB's concurrent_bounded_queue or write a custom thread-safe queue. The channel provides blocking `recv` and non-blocking `try_recv` methods similar to pulling from a queue with synchronization. It's a well-tested abstraction that reduces the chance of synchronization bugs, much as one might prefer to use an existing queue class in C++.

- **UI Library:** Ratatui in Rust serves a similar role as NCurses (or PDCurses, etc.) in C++. It abstracts terminal control, layout, and input handling. A C++ developer might not have used a high-level TUI library, but the concept of drawing text boxes, handling resizing, etc., is comparable. One difference is ratatui provides a declarative layout system (with constraints and percentage splits), which is quite convenient.

- **FFI (Foreign Function Interface):** The interactions with the CAEN hardware libraries are done through functions like `felib_open`, `felib_getvalue`, `felib_sendcommand`, etc. These are likely `extern "C"` bindings to the vendor's C library. Using them in Rust feels similar to calling C library functions in C++. The major difference is safety: these functions are probably marked `unsafe` in Rust (since they operate on raw pointers or FFI), but CLIQ wraps them in safe abstractions (e.g., returning a `Result<FELibReturn>` or similar) so that most of the code doesn't deal with `unsafe` directly. In C++, one would call those functions and check return codes; here it's the same, just through a Rust veneer.

In conclusion, the CLIQ codebase is structured in a clear, modular way: **`main.rs`** for initialization, **`tui.rs`** for the runtime control and concurrency, and **`writer.rs`** for data output. The design choices – using threads for parallel data readout and processing, channels for communication, and buffered file I/O with compression – mirror common

C++ solutions for data acquisition systems. The differences lie mostly in Rust's syntax and safety features, but a C++ developer should be able to map these to familiar concepts. By focusing on these three main components and understanding their roles, a new contributor can navigate the code and start making modifications or improvements with confidence. The current stable version of CLIQ operates as described above, providing a robust foundation for DAQ tasks, and any further development will likely continue along these established design patterns[19][75].

Sources:

- CLIQ GitHub Repository – *README and Source Code (main.rs, tui.rs, writer.rs)* [2][75] (Bo Johnson, 2025)

---

[1] [2] [5] [11] [13] [14] [15] [18] [19] [28] [39] [68] [75] [95] README.md

https://github.com/bojohnson5/cliq/blob/f10044bd5cef85f65b1aaa2c6b9dbbc0c3453919/README.md

[3] [4] [6] [7] [8] [9] [10] [12] main.rs

https://github.com/bojohnson5/cliq/blob/f10044bd5cef85f65b1aaa2c6b9dbbc0c3453919/src/main.rs

[16] [17] [20] [21] [22] [23] [24] [25] [26] [27] [29] [30] [31] [32] [33] [34] [35] [36] [37] [38] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] [51] [52] [53] [54] [55] [56] [57] [58] [59] [60] [61] [62] [63] [64] [65] [91] [92] tui.rs

https://github.com/bojohnson5/cliq/blob/7d4b92484276acb4a6f5919511caf1ea3c4cddb8/src/tui.rs

[66] [67] [69] [70] [71] [72] [73] [74] [76] [77] [78] [79] [80] [81] [82] [83] [84] [85] [86] [87] [88] [89] [90] [93] [94] [96] [97] writer.rs

https://github.com/bojohnson5/cliq/blob/f10044bd5cef85f65b1aaa2c6b9dbbc0c3453919/src/writer.rs